



UNIVERSITÉ
DE LORRAINE



RAPPORT GÉNIE LOGICIEL

Analyse des accidents du travail en France

Projet universitaire
Master 1 MIAGE

Rendu le 22/05/2026

Membres du groupe :

EL AOUDI Rim
FONTANEZ Antoine
FUMERON–LECOMTE Baptiste
KACHLER Théo
MANGIN Raphaël
PIERROT Nathan
SAHRAOUI DOUKKALI Mouad

Table des matières

1. Introduction.....	5
1.1 Contexte et enjeux.....	5
1.2 Objectifs du projet.....	5
1.3 Organisation du rapport.....	6
2. Présentation du projet	7
2.1 Description générale de l'application.....	7
2.2 Jeux de données utilisés	7
2.3 Public cible	8
2.4 Fonctionnalités principales	8
3. Organisation du projet.....	9
3.1 Présentation de l'équipe	9
3.2 Méthode de travail.....	9
3.3 Découpage du développement.....	10
3.4 Gestion de version.....	10
4. Installation et utilisation	11
4.1 Prérequis.....	11
4.2 Installation	11
4.3 Lancement	11
5. Conception de l'application	12
5.1 Architecture générale	12
5.2 Diagramme de cas d'utilisation	12
5.3 Diagramme de classes / données.....	13
5.4 Choix de conception importants	15
5.5 Challenges de conception	16
6. Description technique	17
6.1 Technologies utilisées	17
6.2 Traitement des données	17
6.3 Indicateurs utilisés.....	18
6.4 Intégration continue.....	18
7. Qualité du code et bonnes pratiques	19
7.1 Séparation des responsabilités	19

7.2 Lisibilité et maintenabilité	19
7.3 Robustesse du parsing.....	20
7.4 Documentation	20
7.5 Limites de qualité identifiées	20
8. Tests et validation.....	21
8.1 Stratégie de test.....	21
8.2 Tests unitaires.....	21
8.3 Tests de validation manuelle	21
8.4 Monkey tests	22
8.5 Intégration continue et couverture.....	22
8.6 Bilan des tests.....	22
9. Présentation de l'application	23
9.1 Interface générale.....	23
9.2 Onglet Analyse	23
9.3 Onglet Données / graphiques avancés.....	24
9.4 Onglet Rapports.....	25
9.5 Expérience utilisateur	25
10. Analyse des résultats et pertinence des visualisations	26
10.1 Intérêt de l'analyse	26
10.2 Choix des visualisations	26
10.3 Pertinence métier.....	26
10.4 Limites de l'analyse	27
11. Difficultés rencontrées et solutions trouvées	28
11.1 Difficultés techniques.....	28
11.2 Difficultés humaines et organisationnelles	28
11.3 Solutions mises en place	28
12. Perspectives d'évolution	29
12.1 Enrichissement des données	29
12.2 Carte interactive	29
12.3 Export des résultats.....	29
12.4 Amélioration des tests	29
12.5 Amélioration de l'interface	29

13. Conclusion	30
14. Annexes	31
Sources de données	31
Dépôt Git.....	31
Captures d'écran.....	32
Glossaire	35

1. Introduction

1.1 Contexte et enjeux

Dans le cadre de notre projet, nous nous sommes intéressés à la prévention des risques professionnels, car il s'agit d'un sujet important aussi bien pour les salariés que pour les entreprises. Chaque année en France, de nombreux accidents du travail sont déclarés. Ces accidents peuvent avoir des conséquences lourdes : arrêts de travail, incapacités temporaires ou permanentes, désorganisation dans les entreprises, coûts supplémentaires pour les organismes concernés, et dans les cas les plus graves, décès.

Les données sur les accidents du travail sont disponibles en open data, notamment grâce aux publications de l'Assurance Maladie, de la DARES et de l'INSEE. Cependant, elles ne sont pas toujours simples à utiliser directement. Elles se présentent souvent sous forme de fichiers assez volumineux, avec beaucoup de colonnes, des termes techniques et des nomenclatures comme les codes CTN ou NAF. Pour une personne qui ne connaît pas déjà ces formats, il peut donc être difficile de comprendre rapidement les informations importantes.

Notre projet part donc de ce constat : les données existent, mais elles sont difficiles à lire et à interpréter telles quelles. Nous avons donc voulu développer une application capable de transformer ces données brutes en informations plus claires, grâce à des graphiques et des outils de filtrage. L'objectif n'est pas simplement de présenter des statistiques, mais de faciliter l'analyse des accidents du travail afin d'identifier les secteurs les plus à risque, de comparer différentes situations et de mieux comprendre les principales causes d'accidents.

1.2 Objectifs du projet

L'objectif principal du projet est de développer une application Java permettant d'analyser les accidents du travail en France à partir de données ouvertes. L'application doit offrir une interface claire, permettre le filtrage des données et proposer des représentations graphiques adaptées à différents types d'analyse.

Les objectifs fonctionnels retenus sont les suivants :

- charger et exploiter des fichiers de données au format Excel ;
- afficher les statistiques d'accidents du travail par secteur d'activité ;
- permettre le filtrage par année, secteur CTN, niveau de granularité NAF et indicateur ;
- proposer plusieurs types de visualisations : classement, comparaison, répartition, causes, fréquence, gravité ;
- calculer ou représenter des indicateurs utiles à l'analyse des risques professionnels ;
- rendre l'application suffisamment intuitive pour être utilisée par un public non technique.

Au-delà du résultat applicatif, le projet devait également répondre aux attentes du module de génie logiciel : organisation du travail en équipe, utilisation de Git, structuration du code, documentation, tests, validation et réflexion sur les choix de conception.

1.3 Organisation du rapport

Ce rapport est organisé de manière à présenter les différentes étapes de notre projet. Dans un premier temps, nous expliquons le contexte du sujet ainsi que les données utilisées. Ensuite, nous présentons l'organisation du groupe et notre méthode de travail. Nous détaillons également les choix de conception, les technologies employées, la qualité du code et les tests réalisés. Une partie est ensuite consacrée à l'application finale, avec une présentation des visualisations proposées et de leur intérêt pour l'analyse. Enfin, nous revenons sur les difficultés rencontrées, les améliorations possibles et ce que ce projet nous a apporté.

2. Présentation du projet

2.1 Description générale de l'application

L'application que nous avons développée est un outil d'aide à l'analyse des accidents du travail. Elle a pour but de faciliter la lecture des données liées aux risques professionnels grâce à une interface graphique réalisée avec JavaFX.

L'utilisateur peut choisir une année, sélectionner un ou plusieurs secteurs CTN, modifier le niveau de détail des données avec la nomenclature NAF et choisir l'indicateur qu'il souhaite observer. Les graphiques sont ensuite mis à jour automatiquement afin de présenter les résultats de manière visuelle et plus compréhensible.

L'intérêt de l'application est donc de rendre des données complexes plus accessibles. Au lieu de consulter directement des fichiers Excel difficiles à lire, l'utilisateur peut analyser les accidents du travail sous plusieurs angles : nombre total d'accidents, répartition par secteur, évolution dans le temps, principales causes d'accidents, comparaison entre fréquence et gravité, ou encore analyse plus détaillée des sous-secteurs NAF.

2.2 Jeux de données utilisés

Les données utilisées dans notre application proviennent principalement des fichiers open data de l'Assurance Maladie, dans la partie dédiée aux risques professionnels. Pour notre projet, nous avons travaillé sur les années 2020, 2021 et 2023. L'année 2022 n'était pas disponible dans les fichiers que nous avons exploités, ce qui limite un peu l'analyse de l'évolution dans le temps.

Les fichiers contiennent des informations organisées par secteur d'activité. Chaque ligne correspond à un croisement entre un secteur CTN et un code NAF. On y retrouve notamment des informations sur les secteurs, comme le code CTN, le libellé CTN ou encore les différents niveaux de codes NAF. Les fichiers indiquent aussi des données liées aux effectifs, comme le nombre de salariés, les heures travaillées ou le nombre de SIRET.

Ils contiennent également plusieurs indicateurs liés aux accidents du travail : nombre d'accidents, accidents avec arrêt, incapacités permanentes, décès ou encore journées d'incapacité temporaire. Enfin, les données permettent aussi d'identifier certaines causes d'accidents, comme la manutention manuelle, les chutes, les machines, les risques chimiques, les risques routiers ou les agressions.

L'utilisation des codes NAF est importante, car elle permet d'analyser les données à plusieurs niveaux. On peut donc avoir une vision globale par grands secteurs d'activité, mais aussi aller plus loin avec une analyse plus détaillée par sous-secteurs.

2.3 Public cible

L'application peut être utile à différents profils d'utilisateurs. Elle peut par exemple servir à des professionnels des ressources humaines qui souhaitent mieux connaître les risques associés à leur secteur d'activité. Elle peut aussi intéresser des acteurs de la prévention des risques professionnels, des chercheurs, des étudiants ou toute personne voulant mieux comprendre les différences d'accidents du travail selon les secteurs.

Nous avons choisi de développer une interface graphique plutôt qu'un simple script ou un tableau de résultats, car l'objectif était de rendre l'analyse plus accessible. L'utilisateur peut ainsi explorer les données plus facilement, sans avoir besoin de manipuler directement les fichiers sources.

2.4 Fonctionnalités principales

Les fonctionnalités principales de l'application sont les suivantes :

1. Filtres dynamiques
L'utilisateur peut filtrer les données par année, secteur CTN, niveau NAF et indicateur mesuré. Ces filtres permettent de personnaliser l'analyse selon le besoin.
2. Classement des secteurs
Un graphique en barres permet d'identifier rapidement les secteurs les plus concernés par l'indicateur sélectionné.
3. Comparaison des accidents par secteur et par année
L'application permet de comparer certains secteurs sur plusieurs années afin d'observer les évolutions.
4. Répartition des accidents par secteur
Un graphique de type camembert donne une vue globale du poids relatif des différents secteurs dans l'ensemble des accidents.
5. Analyse des causes d'accidents
Des graphiques empilés permettent de visualiser les principales causes d'accidents pour les secteurs sélectionnés.
6. Top sous-secteurs NAF
L'application identifie les sous-secteurs les plus touchés à l'intérieur d'un secteur donné.
7. Fréquence vs gravité
Un nuage de points positionne les secteurs selon leur indice de fréquence et leur taux de gravité, ce qui permet de distinguer les secteurs ayant beaucoup d'accidents de ceux où les accidents sont particulièrement graves.
8. Vue multicritères
Une comparaison normalisée permet d'observer plusieurs dimensions de risque : nombre d'accidents, fréquence, gravité et décès.

3. Organisation du projet

3.1 Présentation de l'équipe

Le projet a été réalisé par une équipe de sept étudiants. La répartition des rôles a été effectuée en fonction des compétences, des préférences et des besoins techniques du projet.

Membre	Rôle principal
Antoine FONTANEZ	Référent Git, configuration Maven, DevOPS
Baptiste FUMERON-LECOMTE	Parsing Excel, accès aux données, repository
Théo KACHLER	Services métier, calculs statistiques, agrégations
Raphaël MANGIN	Tests JUnit, documentation
Nathan PIERROT	Interface JavaFX, navigation, intégration des onglets
Mouad SAHRAOUI DOUKKALI	Couche graphique avec JFreeChart
Rim EL AOUDI	Rapport, maquettes, contribution aux tests

3.2 Méthode de travail

Nous avons choisi de nous appuyer sur une approche proche du modèle en cascade, car elle correspondait assez bien à notre façon de travailler sur ce projet.

Au départ, les grandes fonctionnalités à réaliser étaient plutôt claires :

- charger les données ;
- permettre à l'utilisateur de les filtrer ;
- calculer les indicateurs utiles ;
- afficher les résultats sous forme de graphiques lisibles.

Cette organisation nous a permis d'avancer étape par étape, sans partir directement dans le développement. Nous avons d'abord défini les besoins dans un cahier des charges, puis nous avons réfléchi à la structure générale de l'application.

Le projet a ensuite été découpé en plusieurs parties :

- la gestion des données ;
- la logique de traitement ;
- la création des graphiques ;
- l'interface utilisateur.

Ce découpage nous a aidés à mieux répartir le travail dans le groupe et à garder une vision claire de l'avancement.

Nous avons aussi effectué des vérifications sur les éléments essentiels, notamment le chargement des données, l'affichage des résultats et le bon fonctionnement des graphiques. Enfin, l'interface a été testée manuellement afin de vérifier que l'application restait compréhensible et utilisable.

Cette méthode nous a donc aidés à rester organisés tout au long du projet. Elle était aussi adaptée au temps limité dont nous disposions, car elle nous a permis de nous concentrer d'abord sur les fonctionnalités indispensables avant de penser aux améliorations secondaires.

3.3 Découpage du développement

Le développement a été organisé en plusieurs étapes successives.

Étape	Objectifs	Livrables attendus
Étape 1	Mise en place du projet	Projet Maven, structure Git, premières classes métier
Étape 2	Chargement des données	Parser Excel, repository, premiers tests
Étape 3	Services métier	Calculs statistiques, agrégations, filtres
Étape 4	Interface	Fenêtre JavaFX, onglets, panneau de filtres
Étape 5	Graphiques	Visualisations principales et avancées
Étape 6	Finalisation	Tests, documentation, rapport, préparation de la démonstration

3.4 Gestion de version

Le projet a été géré avec Git et hébergé sur GitHub. Cela nous a permis de travailler plus facilement à plusieurs, de garder un historique des modifications et de sécuriser le code en cas d'erreur ou de mauvaise manipulation.

Nous avons organisé le dépôt avec différents dossiers afin de séparer les parties du projet : lecture des données, services métier, interface, graphiques et tests. Le travail a aussi été réparti à l'aide de branches, ce qui permettait à chaque membre d'avancer sur une fonctionnalité sans modifier directement la branche principale.

Git a donc été un outil important, pas seulement pour le code, mais aussi pour l'organisation du groupe. Il nous a obligés à mieux coordonner nos changements, à communiquer sur les parties développées par chacun et à résoudre les problèmes d'intégration lorsque plusieurs modules devaient fonctionner ensemble.

4. Installation et utilisation

4.1 Prérequis

Pour lancer l'application, il est nécessaire de disposer des éléments suivants :

- un JDK Java compatible avec le projet ;
- Maven 3.9 ou supérieur ;
- un système Windows, macOS ou Linux ;
- un accès au dépôt GitHub du projet.

4.2 Installation

L'installation se fait à partir du dépôt Git :

```
git clone https://github.com/Akip2/genie-logiciel.git  
cd genie-logiciel/project  
mvn clean install
```

Les fichiers de données nécessaires au lancement de l'application sont inclus dans le dossier [project/data/](#)

4.3 Lancement

Depuis le dossier [project/](#), l'application peut être lancée avec la commande suivante :

```
mvn exec:java
```

La classe principale exécutée est [com.example.testfx.Main](#) . Au lancement, la fenêtre principale s'ouvre avec l'onglet d'analyse sélectionné par défaut.

5. Conception de l'application

5.1 Architecture générale

L'application a été conçue avec une architecture en couches afin de séparer clairement les responsabilités. Cette organisation permet d'éviter de regrouper toute la logique dans une seule classe et rend le code plus lisible, plus maintenable et plus facile à faire évoluer.

Elle a aussi facilité le travail en équipe, car chaque membre pouvait avancer sur une partie précise du projet, comme la lecture des données, la logique métier, les graphiques ou l'interface, tout en conservant une structure globale cohérente entre les différents modules.

Nous avons distingué plusieurs parties principales :

- la couche parsing et données : elle s'occupe de lire les fichiers Excel, de récupérer les informations utiles et de les transformer en objets Java exploitables par l'application ;
- la couche métier : elle contient les traitements principaux, comme le filtrage des données, le calcul des indicateurs et l'agrégation des résultats ;
- la couche graphique : elle utilise les résultats calculés pour générer les différents graphiques affichés dans l'application ;
- la couche interface utilisateur : elle gère l'affichage global de l'application, les onglets, les filtres et l'intégration des graphiques.

5.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les principales interactions entre l'utilisateur, le système et l'application d'aide à la décision.

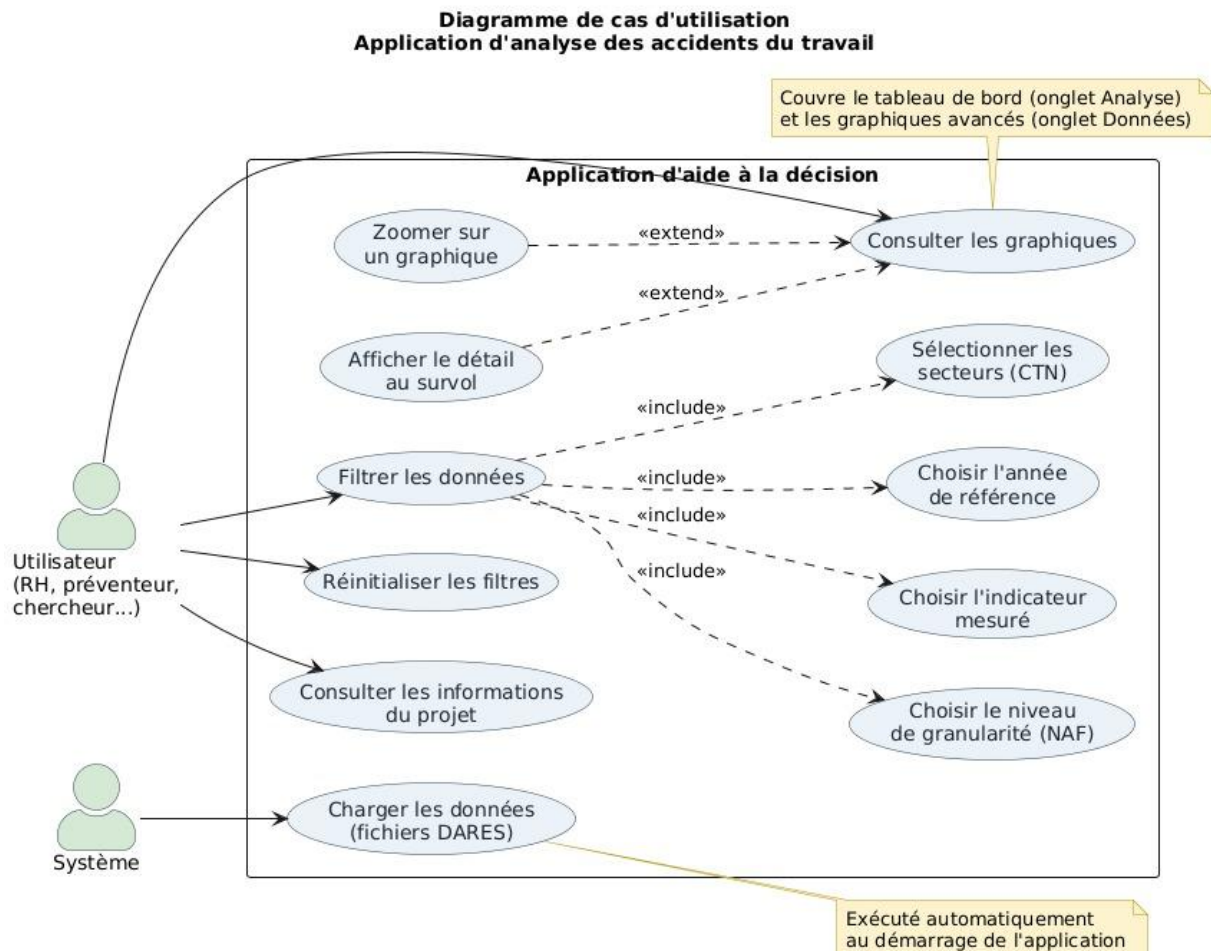
L'acteur principal est l'utilisateur. Il peut s'agir par exemple d'un professionnel RH, d'un acteur de la prévention, d'un chercheur ou d'un étudiant. Son objectif est de consulter, filtrer et analyser les données sur les accidents du travail afin de mieux comprendre les risques selon les années, les secteurs et les indicateurs disponibles.

Le diagramme montre également un second acteur, le système, qui intervient automatiquement au démarrage de l'application pour charger les données issues des fichiers DARES.

Les principales fonctionnalités accessibles à l'utilisateur sont :

- filtrer les données, en choisissant l'année de référence, les secteurs CTN, l'indicateur mesuré et le niveau de granularité NAF ;
- consulter les graphiques, aussi bien dans le tableau de bord principal que dans les graphiques avancés ;
- zoomer sur un graphique, afin d'analyser plus précisément certaines valeurs ;

- afficher les détails au survol, pour obtenir des informations complémentaires sur un point ou une barre du graphique ;
- réinitialiser les filtres, afin de revenir rapidement à une vue globale ;
- consulter les informations du projet, comme le contexte, les objectifs et le public cible.



5.3 Diagramme de classes / données

Le diagramme de classes présente l'organisation générale de l'application et les principales relations entre les composants. Il montre que le projet est structuré autour de plusieurs parties : l'interface, le contrôleur, les services, le dépôt de données, le parser et les modèles de données.

La classe App représente le point d'entrée de l'application. Elle crée la vue principale DashboardView, qui correspond à l'interface utilisateur. Cette vue utilise une ChartFactory pour générer les différents graphiques affichés à l'écran.

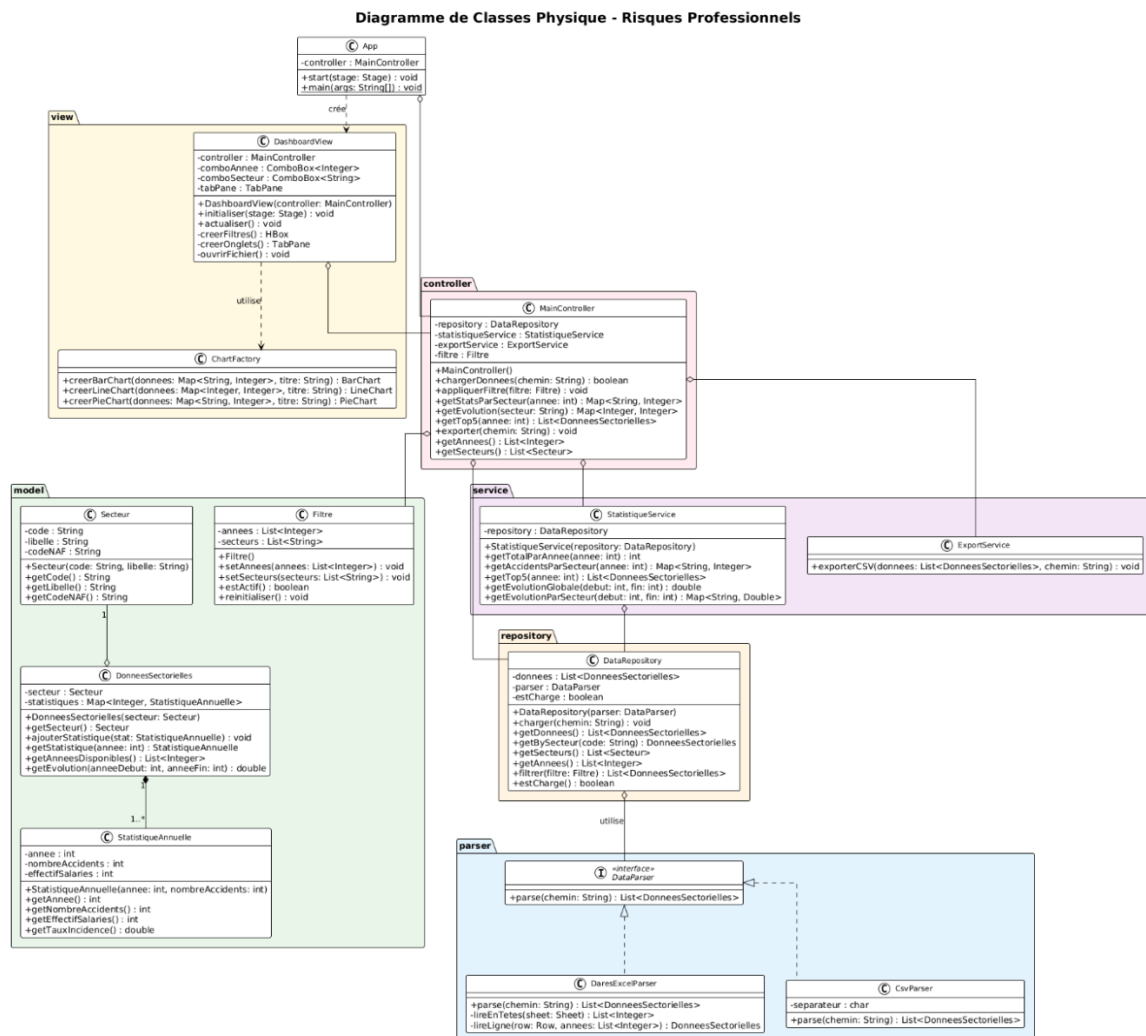
Le MainController joue un rôle central dans l'application. Il fait le lien entre l'interface graphique, les données et les services. Il récupère les choix de l'utilisateur, comme les filtres ou les années sélectionnées, puis appelle les services nécessaires pour produire les résultats à afficher.

Les données sont centralisées dans la classe DataRepository. Celle-ci s'appuie sur un DataParser pour charger les fichiers de données. Le diagramme montre plusieurs implémentations possibles du parser, comme DaresExcelParser pour les fichiers Excel et CsvParser pour les fichiers CSV. Cette organisation permet de rendre le chargement des données plus flexible.

La partie métier est représentée par StatistiqueService, qui calcule les indicateurs utiles à l'analyse : total des accidents, top des secteurs, évolution globale ou encore évolution par secteur. Le service ExportService permet quant à lui de prévoir l'export des données au format CSV.

Enfin, les classes du modèle, comme Secteur, Filtre, DonneesSectorielles et StatistiqueAnnuelle, représentent les informations manipulées par l'application. Elles permettent de structurer les données avant leur traitement et leur affichage.

Ce diagramme montre donc une conception organisée et modulaire. Chaque classe a une



responsabilité précise, ce qui rend l'application plus lisible, plus maintenable et plus facile à faire évoluer.

5.4 Choix de conception importants

Un premier choix important a été de récupérer les colonnes Excel à partir de leur nom et non de leur position dans le fichier. Ce choix est utile, car l'ordre des colonnes peut changer d'une année à l'autre. Grâce à cette solution, le parser est plus fiable : si une colonne est déplacée dans le fichier, les données peuvent quand même être lues correctement, sans fausser les résultats.

Nous avons aussi fait le choix de bien séparer la logique métier de la partie graphique. Les services s'occupent uniquement de traiter les données, de calculer les indicateurs et de préparer les résultats. Ensuite, les classes responsables des graphiques utilisent ces résultats pour construire les visualisations. Cette séparation permet d'éviter un couplage trop fort entre les calculs et l'interface.

Enfin, l'utilisation d'un gestionnaire de graphiques permet de simplifier la création des visualisations. L'interface n'a pas besoin de connaître tous les détails techniques de construction des graphiques : elle demande simplement au composant dédié de fournir le graphique nécessaire.

5.5 Challenges de conception

Plusieurs défis de conception ont été rencontrés :

- gestion de données complexes : les fichiers exploités contiennent de nombreuses colonnes, plusieurs indicateurs et différents niveaux de nomenclature, notamment avec les codes CTN et NAF ;
- compréhension du code déjà écrit par un autre membre : pour développer correctement l'interface, il a fallu analyser le code existant afin de comprendre sa structure, les méthodes disponibles, les objets utilisés et les résultats déjà produits ;
- cohérence entre l'interface et le code existant : les choix effectués par l'utilisateur dans l'interface devaient être correctement reliés aux fonctions déjà développées, afin d'obtenir des résultats cohérents avec les filtres sélectionnés.
- choix des indicateurs : il fallait éviter de se limiter au nombre brut d'accidents, car cet indicateur peut être trompeur si l'on ne prend pas en compte la taille des secteurs ou les effectifs concernés ;
- lisibilité des graphiques : les visualisations devaient rester compréhensibles malgré la quantité importante de données à afficher ;
- testabilité : certaines parties, comme l'interface graphique, sont plus difficiles à tester automatiquement que le parser ou les services métier.

6. Description technique

6.1 Technologies utilisées

Le projet repose sur les technologies suivantes :

Technologie	Utilisation
Java	Langage principal du projet
JavaFX	Construction de l'interface graphique
Maven	Gestion du projet, des dépendances et du lancement
Apache POI	Lecture des fichiers Excel
JFreeChart	Création des graphiques
JUnit 5	Tests unitaires
JaCoCo	Mesure de couverture de tests
GitHub Actions	Intégration continue

Ces choix techniques sont cohérents avec les objectifs du projet. Java permet de développer une application structurée et adaptée à un projet universitaire de génie logiciel. JavaFX a été utilisé pour construire une interface graphique claire et interactive. Maven facilite la gestion des dépendances et le lancement du projet.

6.2 Traitement des données

Dans un premier temps, les fichiers Excel sont lus par le parser. Celui-ci récupère les informations utiles du fichier en tenant compte des noms de colonnes. Cela permet d'éviter les erreurs si l'ordre des colonnes change entre deux fichiers. Les lignes inutiles, comme les titres ou les métadonnées, sont ignorées. Ensuite, chaque ligne exploitable est transformée en objet Java afin de pouvoir être utilisée dans l'application.

Dans un deuxième temps, les données chargées sont centralisées dans le `DataRepository`. Cette classe sert de point d'accès principal aux données. Elle permet par exemple de récupérer toutes les données, de filtrer par année, par secteur CTN ou par code NAF, mais aussi d'obtenir la liste des années ou des secteurs disponibles.

Enfin, les services métier utilisent ces données pour produire les résultats nécessaires à l'affichage. Ils réalisent les calculs et les agrégations, comme le total des accidents par secteur, la répartition en pourcentage, l'évolution selon les années, l'analyse des causes, l'indice de fréquence ou encore le taux de gravité. Ces résultats sont ensuite transmis à la partie graphique pour être affichés sous forme de visualisations.

6.3 Indicateurs utilisés

Trois types d'indicateurs sont particulièrement importants :

Indicateur	Rôle
Nombre d'accidents du travail	Il permet de mesurer le volume total d'accidents pour un secteur donné.
Indice de fréquence	Il permet de comparer les secteurs en tenant compte du nombre de salariés.
Taux de gravité	Il met en relation les journées d'incapacité temporaire avec le nombre d'heures travaillées.

Le nombre d'accidents du travail donne une première vision de la situation, mais il ne suffit pas toujours pour comparer correctement les secteurs. En effet, un secteur avec beaucoup de salariés peut avoir naturellement plus d'accidents qu'un secteur plus petit.

C'est pour cela que l'indice de fréquence est particulièrement utile. Il permet de rapporter le nombre d'accidents à l'effectif salarié, ce qui rend la comparaison plus juste entre des secteurs de tailles différentes.

Enfin, le taux de gravité apporte une autre lecture des données. Il ne s'intéresse pas seulement au nombre d'accidents, mais aussi à leurs conséquences, notamment à travers les journées d'incapacité temporaire. Grâce à ces indicateurs, l'application permet donc d'analyser les risques professionnels de manière plus complète.

6.4 Intégration continue

Une intégration continue a été mise en place avec GitHub Actions. À chaque modification envoyée sur le dépôt, notamment lors d'un push ou d'une pull request vers la branche principale, une vérification automatique est lancée.

Cette vérification exécute une build Maven, lance les tests et génère la couverture de tests avec JaCoCo. Cela permet de s'assurer régulièrement que le projet compile correctement et que les tests existants continuent de fonctionner.

Dans un projet de groupe, cette automatisation est importante, car elle limite les risques liés à l'intégration du code de chacun. Elle permet de détecter plus rapidement une erreur introduite par une modification et apporte donc une sécurité supplémentaire avant de valider les changements sur la branche principale.

7. Qualité du code et bonnes pratiques

7.1 Séparation des responsabilités

La qualité du code repose surtout sur une bonne séparation des responsabilités. Le projet est organisé en plusieurs packages, chacun avec un rôle précis :

- parser : lecture et traitement des fichiers de données ;
- data : accès aux données chargées ;
- service : logique métier, calculs et agrégations ;
- model : représentation des principales entités manipulées ;
- dto : objets de transfert entre les différentes couches ;
- chart : création et gestion des graphiques ;
- onglets : construction des différentes vues de l'interface.

Cette organisation rend le projet plus lisible et plus simple à maintenir. Lorsqu'un développeur veut modifier une partie précise, par exemple le chargement des fichiers, les calculs statistiques ou l'affichage graphique, il sait directement dans quel package intervenir. Cela limite aussi les mélanges de responsabilités et facilite l'évolution du projet.

7.2 Lisibilité et maintenabilité

Le code utilise des noms de classes et de méthodes assez explicites. Les principales classes indiquent directement leur rôle dans l'application, par exemple `ExcelParser`, `DataRepository`, `StatisticsServiceImpl`, `ChartManager` ou encore `OngletAnalyse`.

Cette lisibilité est importante dans un projet réalisé en groupe. Elle permet à un membre de l'équipe de comprendre plus facilement une partie du code, même s'il ne l'a pas développée lui-même. Cela facilite aussi la reprise du projet, les corrections et l'intégration entre les différentes parties de l'application.

7.3 Robustesse du parsing

La lecture des fichiers de données a été pensée pour être la plus fiable possible. Un choix important a été d'utiliser les noms des colonnes Excel plutôt que leur position. Ainsi, si l'ordre des colonnes change légèrement d'un fichier à l'autre, le parser peut quand même récupérer les bonnes informations.

Des traitements ont aussi été prévus pour gérer certains cas particuliers, comme les cellules vides, les valeurs numériques ou les formats inattendus. Cette partie est essentielle, car le parser est la base du projet. Si les données sont mal chargées, les calculs, les graphiques et l'analyse finale peuvent être faussés.

7.4 Documentation

Le projet contient un README qui présente les éléments importants : le sujet, les fonctionnalités, l'installation, l'architecture, les données utilisées, les indicateurs et la répartition des rôles dans l'équipe.

Cette documentation est utile pour comprendre rapidement le projet et pour permettre à une autre personne de le lancer plus facilement. Elle est aussi importante dans le cadre de l'évaluation, car elle montre que le projet ne se limite pas au code, mais qu'il est aussi expliqué et structuré.

Certaines parties du code contiennent également des commentaires, notamment pour expliquer des choix de parsing ou certains calculs métier. Les commentaires ne remplacent pas un code clair, mais ils permettent de mieux comprendre les décisions prises sur les parties plus sensibles.

7.5 Limites de qualité identifiées

Même si le projet est structuré, certaines améliorations restent possibles. La couverture de tests pourrait être étendue, notamment sur les services métier et sur certains comportements liés aux graphiques.

Il serait aussi possible d'ajouter plus de contrôles pour mieux informer l'utilisateur lorsqu'un fichier est incorrect, lorsqu'une donnée est manquante ou lorsqu'un filtre ne retourne aucun résultat.

Enfin, certaines parties de l'interface pourraient être davantage factorisées si le projet était poursuivi. Cela permettrait de réduire les répétitions et de rendre l'application encore plus facile à maintenir et à faire évoluer.

8. Tests et validation

8.1 Stratégie de test

Notre stratégie de test a été de vérifier en priorité les parties les plus importantes pour le bon fonctionnement de l'application. Nous avons donc commencé par tester le parser Excel et le repository de données, car ce sont les éléments de base du projet.

En effet, si les données ne sont pas correctement lues ou stockées, les calculs, les indicateurs et les graphiques risquent d'être incorrects. Il était donc essentiel de s'assurer que cette partie fonctionnait correctement.

En complément des tests automatiques, nous avons aussi réalisé des tests manuels sur l'interface. Ces vérifications nous ont permis de contrôler que les filtres fonctionnaient, que les graphiques se mettaient bien à jour et que l'application restait utilisable dans des cas d'utilisation réels.

8.2 Tests unitaires

Les tests unitaires ont été réalisés avec JUnit. Ils concernent principalement deux parties du projet :

- ExcelParser : nous avons testé la lecture d'un fichier valide, mais aussi plusieurs cas d'erreur comme un nom de fichier invalide, un fichier inexistant, un mauvais format, un fichier mal structuré ou encore un fichier vide ;
- DataRepository : nous avons vérifié le chargement des données, la récupération des années disponibles, les filtres par année, par CTN, par code NAF ou NAF2, ainsi que la recherche par activité.

Ces tests permettent de couvrir à la fois des situations normales et des cas d'erreur. L'objectif était de vérifier que l'application fonctionne avec des données correctes, mais aussi qu'elle réagit de manière contrôlée lorsqu'elle reçoit des données invalides ou incomplètes.

8.3 Tests de validation manuelle

Les tests de validation manuelle ont consisté à lancer l'application et à l'utiliser comme un utilisateur final. Nous avons testé les différents filtres disponibles : année, secteur CTN, niveau de granularité NAF et indicateur sélectionné.

Ces tests nous ont permis de vérifier plusieurs points :

- l'affichage correct de l'onglet Analyse ;
- la mise à jour des graphiques après un changement de filtre ;
- la cohérence générale des résultats affichés ;
- la lisibilité des graphiques ;
- la navigation entre les onglets Analyse, Données et Rapports.

Ces validations étaient importantes, car certaines erreurs ne sont visibles qu'en utilisant réellement l'application.

8.4 Monkey tests

Nous avons aussi réalisé des monkey tests. Le principe est d'utiliser l'application de manière libre, parfois imprévue, afin de repérer des comportements anormaux.

Ces tests ont permis d'identifier un problème lié au bouton de réinitialisation. Celui-ci réinitialisait certains éléments de l'affichage, mais ne remettait pas toujours à zéro le zoom appliqué sur certains graphiques.

Ce type de test est intéressant, car il permet de détecter des problèmes liés à l'expérience utilisateur. Ces erreurs ne sont pas forcément repérées par les tests unitaires, qui se concentrent plutôt sur des méthodes ou des composants précis.

8.5 Intégration continue et couverture

L'intégration continue a été mise en place avec GitHub Actions. Elle permet de lancer automatiquement la compilation Maven, les tests et la génération du rapport de couverture avec JaCoCo.

Ce mécanisme est utile dans un projet de groupe, car il permet de vérifier régulièrement que l'application compile toujours correctement après les modifications. Il aide aussi à repérer rapidement les erreurs introduites lors de l'intégration du code.

La couverture JaCoCo donne une indication sur les parties du code exécutées par les tests. Elle ne garantit pas que le projet ne contient aucun bug, mais elle permet d'identifier les zones moins testées et donc les parties à améliorer en priorité.

8.6 Bilan des tests

Les tests réalisés ont permis de valider les parties les plus importantes du projet, notamment le chargement et l'accès aux données. Ils ont aussi permis de repérer certains comportements inattendus lors de l'utilisation de l'interface.

La principale limite concerne les tests de l'interface graphique, qui sont plus difficiles à automatiser que les tests sur le parser ou le repository. Pour améliorer cette partie, il serait possible d'ajouter par la suite des tests d'intégration ou des tests d'interface avec des outils spécialisés. Cela permettrait de vérifier automatiquement certains scénarios utilisateur, comme le changement de filtres ou la mise à jour des graphiques.

9. Présentation de l'application

9.1 Interface générale

L'application se présente sous la forme d'une interface de bureau. Elle est organisée avec un bandeau supérieur, un panneau de filtres à gauche et une zone principale dédiée à l'affichage des graphiques.

Le bandeau supérieur permet de naviguer entre trois onglets :

- Analyse ;
- Données ;
- Rapports.

Le panneau de gauche regroupe les principaux paramètres d'analyse : l'année de référence, les secteurs CTN, le niveau de granularité, l'indicateur à observer et le bouton de mise à jour.

Cette organisation permet à l'utilisateur de garder les filtres visibles pendant toute l'utilisation de l'application. Il peut ainsi modifier les paramètres et observer directement les résultats dans la zone centrale.

9.2 Onglet Analyse

L'onglet Analyse correspond au tableau de bord principal de l'application. Il regroupe plusieurs graphiques complémentaires afin de donner une vue d'ensemble des accidents du travail.

On y retrouve notamment :

- un classement des secteurs ;
- une comparaison des accidents par secteur ;
- une répartition des accidents ;
- une analyse des causes d'accidents.

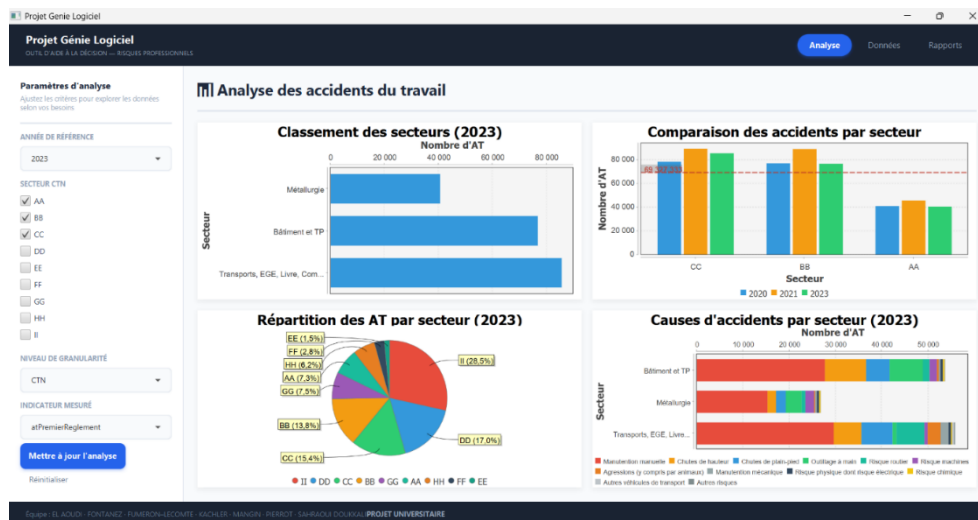


Figure 1 — Tableau de bord principal d'analyse des accidents du travail

9.3 Onglet Données / graphiques avancés

L'onglet Données propose des visualisations plus détaillées :

- top 10 des sous-secteurs ;
- comparaison fréquence / gravité ;
- comparaison multicritères ;
- top des facteurs de risque.



Figure 2 — Graphiques avancés et comparaison multicritères

Cette vue permet d'aller plus loin qu'une simple lecture du nombre d'accidents. Elle met en évidence des différences de profil entre secteurs : certains peuvent avoir beaucoup d'accidents, d'autres peuvent être moins fréquents mais plus graves.

9.4 Onglet Rapports

L'onglet Rapports présente le contexte du projet, les objectifs, les fonctionnalités principales et le public cible.

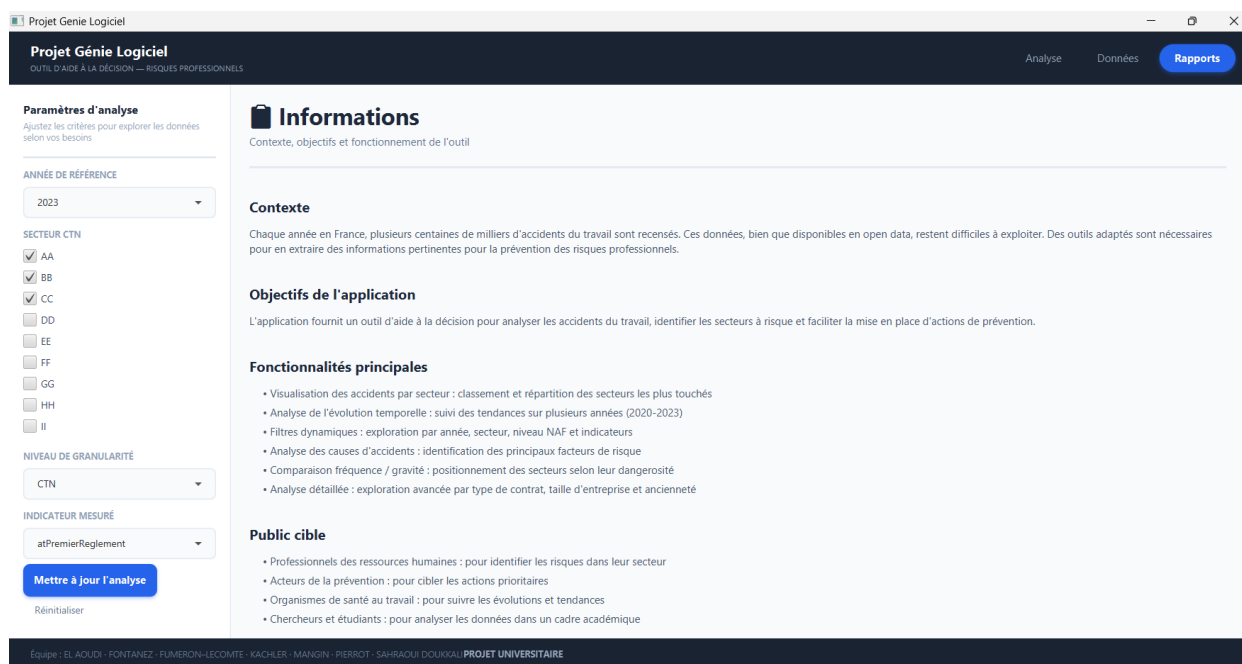


Figure 3 — Onglet d'information et de contextualisation du projet

9.5 Expérience utilisateur

L'interface a été conçue pour rester simple et compréhensible. Les filtres sont regroupés dans une zone unique, ce qui permet à l'utilisateur de modifier facilement ses critères d'analyse. Les onglets sont clairement séparés et les graphiques sont organisés selon le type d'information affichée.

Le choix d'une interface graphique était important pour notre projet. L'objectif était de rendre les données plus accessibles, sans obliger l'utilisateur à ouvrir les fichiers Excel ou à comprendre directement la structure des colonnes. Grâce aux filtres et aux visualisations, il peut obtenir rapidement des informations utiles et comparer les résultats de manière plus intuitive.

10. Analyse des résultats et pertinence des visualisations

10.1 Intérêt de l'analyse

L'application permet de faire ressortir les différences entre les secteurs d'activité. Les accidents du travail ne sont pas répartis de la même manière partout : certains secteurs présentent un nombre élevé d'accidents, tandis que d'autres se distinguent plutôt par la gravité des accidents déclarés.

L'intérêt de l'outil est donc de ne pas se limiter à une seule lecture des données. Le nombre brut d'accidents permet d'identifier les secteurs les plus touchés en volume. L'indice de fréquence permet une comparaison plus juste, car il prend en compte le nombre de salariés dans chaque secteur. Enfin, le taux de gravité apporte une information complémentaire en montrant l'impact des accidents à travers les journées d'incapacité temporaire. Ainsi, l'application permet une analyse plus complète des risques professionnels.

10.2 Choix des visualisations

Les visualisations ont été choisies en fonction des besoins d'analyse. L'objectif était de proposer des graphiques adaptés aux informations que l'utilisateur souhaite observer.

Le graphique en barres est utilisé pour classer les secteurs et comparer rapidement les valeurs. Le graphique circulaire permet de visualiser la répartition globale des accidents entre les secteurs. Le graphique empilé est utile pour comparer les principales causes d'accidents selon les secteurs. Enfin, le nuage de points fréquence / gravité permet d'identifier les secteurs qui présentent à la fois une fréquence élevée d'accidents et une gravité importante.

Ces choix permettent de rendre l'analyse plus claire. Au lieu de présenter uniquement des tableaux de chiffres, l'application aide l'utilisateur à repérer plus facilement les tendances, les écarts et les secteurs qui nécessitent une attention particulière.

10.3 Pertinence métier

La pertinence de l'analyse repose sur la distinction entre le volume, la fréquence et la gravité des accidents. Un secteur peut avoir beaucoup d'accidents simplement parce qu'il emploie un grand nombre de salariés. Dans ce cas, le nombre brut d'accidents ne suffit pas toujours pour évaluer correctement le niveau de risque.

À l'inverse, un secteur plus petit peut avoir moins d'accidents en valeur absolue, mais un indice de fréquence plus élevé. Cela signifie que le risque est plus important lorsqu'on le rapporte à la taille du secteur.

Cette distinction est importante dans une logique de prévention. Elle permet de ne pas se concentrer uniquement sur les secteurs les plus grands ou les plus visibles, mais aussi d'identifier ceux où le risque relatif est le plus élevé.

10.4 Limites de l'analyse

Plusieurs limites doivent être prises en compte.

D'abord, les données utilisées ne couvrent pas toutes les années de manière continue, car l'année 2022 est absente. Cela limite l'analyse temporelle.

Ensuite, l'application repose sur les données disponibles dans les fichiers sources. Si certaines informations sont absentes ou imprécises, l'application ne peut pas les corriger automatiquement.

Enfin, l'outil ne réalise pas de prédiction statistique. Il permet une analyse descriptive, utile pour comprendre et comparer, mais il ne produit pas de modèle prédictif sur les accidents futurs.

11. Difficultés rencontrées et solutions trouvées

11.1 Difficultés techniques

Une première difficulté concernait l'intégration entre le code existant et le front-end. Pour développer correctement l'interface, il a fallu comprendre ce qui avait déjà été réalisé dans le code existant : les données disponibles, les méthodes existantes et les résultats fournis par les services. Cela a demandé un temps d'adaptation afin d'utiliser correctement tout ce qui était déjà en place.

Nous avons aussi dû faire attention à la cohérence entre les différentes couches du projet. Les graphiques dépendaient des résultats calculés par les services métier, eux-mêmes liés au parser et au repository. Pour éviter les confusions, nous avons gardé une séparation claire des responsabilités.

Une autre difficulté technique a été liée à la compatibilité de Java. Au départ, le projet utilisait Java 25, mais cette version a posé des problèmes de compatibilité selon les environnements de travail des membres du groupe. Pour rendre le projet plus stable et plus facilement exécutable par tous, nous avons décidé de passer l'ensemble du projet en Java 21.

11.2 Difficultés humaines et organisationnelles

La principale difficulté humaine a été la coordination entre les différentes parties du projet. Chaque membre avançait sur un module spécifique, mais ces modules devaient ensuite fonctionner ensemble. Il a donc été nécessaire de communiquer régulièrement sur les formats de données attendus, les noms des indicateurs et les besoins de l'interface.

11.3 Solutions mises en place

Les solutions principales ont été :

- structurer le projet en packages clairs ;
- définir des rôles par membre ;
- prioriser les fonctionnalités essentielles ;
- utiliser Git pour suivre les modifications ;
- réaliser des tests unitaires sur les parties sensibles ;
- valider manuellement l'interface avec des scénarios utilisateur.

12. Perspectives d'évolution

12.1 Enrichissement des données

L'application pourrait intégrer davantage d'années afin de proposer une analyse temporelle plus complète. L'ajout de nouvelles sources, par exemple des données INSEE plus détaillées ou des données régionales, permettrait également d'enrichir l'analyse.

12.2 Carte interactive

Une évolution intéressante serait l'ajout d'une carte de France interactive. Elle permettrait de visualiser les accidents du travail par région ou département, afin d'ajouter une dimension géographique à l'analyse.

12.3 Export des résultats

L'export CSV ou PDF des résultats filtrés pourrait faciliter l'utilisation de l'application dans un contexte professionnel ou académique. Un utilisateur pourrait ainsi conserver les graphiques ou les données correspondant à son analyse.

12.4 Amélioration des tests

Les tests pourraient être étendus aux services métier et à l'interface graphique. Des tests d'intégration permettraient de vérifier l'enchaînement complet entre chargement des données, calcul des indicateurs et affichage.

12.5 Amélioration de l'interface

L'interface pourrait être rendue plus interactive, avec des infobulles, des explications sur les indicateurs, des messages d'erreur plus détaillés et une meilleure gestion du zoom sur les graphiques.

13. Conclusion

Ce projet nous a permis de concevoir et de développer une application d'analyse des accidents du travail en France. À partir de données ouvertes complexes, nous avons créé un outil capable de charger les données, de les filtrer, de calculer des indicateurs et de les afficher sous forme de graphiques.

L'application répond aux objectifs principaux fixés au départ. Elle permet d'explorer les données par année, par secteur, par niveau NAF et par indicateur. Elle propose aussi plusieurs visualisations complémentaires afin de comparer les accidents selon leur volume, leur fréquence et leur gravité. L'interface graphique rend ces informations plus accessibles pour un utilisateur qui ne connaît pas forcément la structure des fichiers sources.

Sur le plan du génie logiciel, ce projet nous a permis de mettre en pratique plusieurs notions importantes : le travail en équipe, l'utilisation de Git, l'architecture en couches, la séparation des responsabilités, la documentation, les tests unitaires et la validation manuelle de l'application.

Le résultat final constitue une base fonctionnelle, mais encore évolutive. Certaines améliorations pourraient être ajoutées par la suite, comme une analyse géographique, l'export des résultats ou davantage de tests sur l'interface graphique.

Enfin, ce projet nous a montré qu'un développement logiciel ne repose pas seulement sur le code. L'organisation du groupe, la communication, les choix de conception et la validation progressive sont aussi essentiels pour obtenir une application cohérente et exploitable.

14. Annexes

Sources de données

- Assurance Maladie - Risques professionnels et sinistralité
<https://www.assurance-maladie.ameli.fr/etudes-et-donnees/par-theme/risques-professionnels-et-sinistralite>
- DARES - Accidents du travail
<https://dares.travail-emploi.gouv.fr/donnees/les-accidents-du-travail>
- INSEE - Nomenclature d'activités française
<https://www.insee.fr/fr/information/2120875>

Dépôt Git

Dépôt du projet :

<https://github.com/Akip2/genie-logiciel>

Captures d'écran

Figure - Maquette Tableau de bord principal d'analyse des accidents du travail

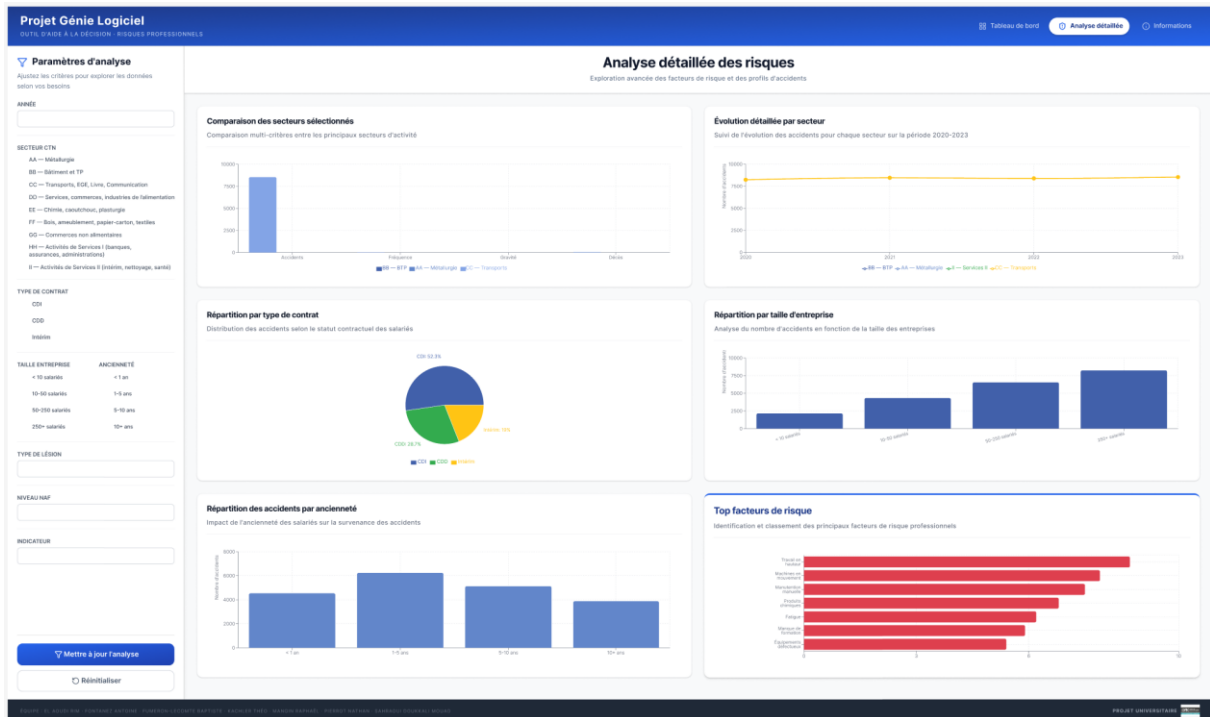


Figure 1 - Tableau de bord principal d'analyse des accidents du travail

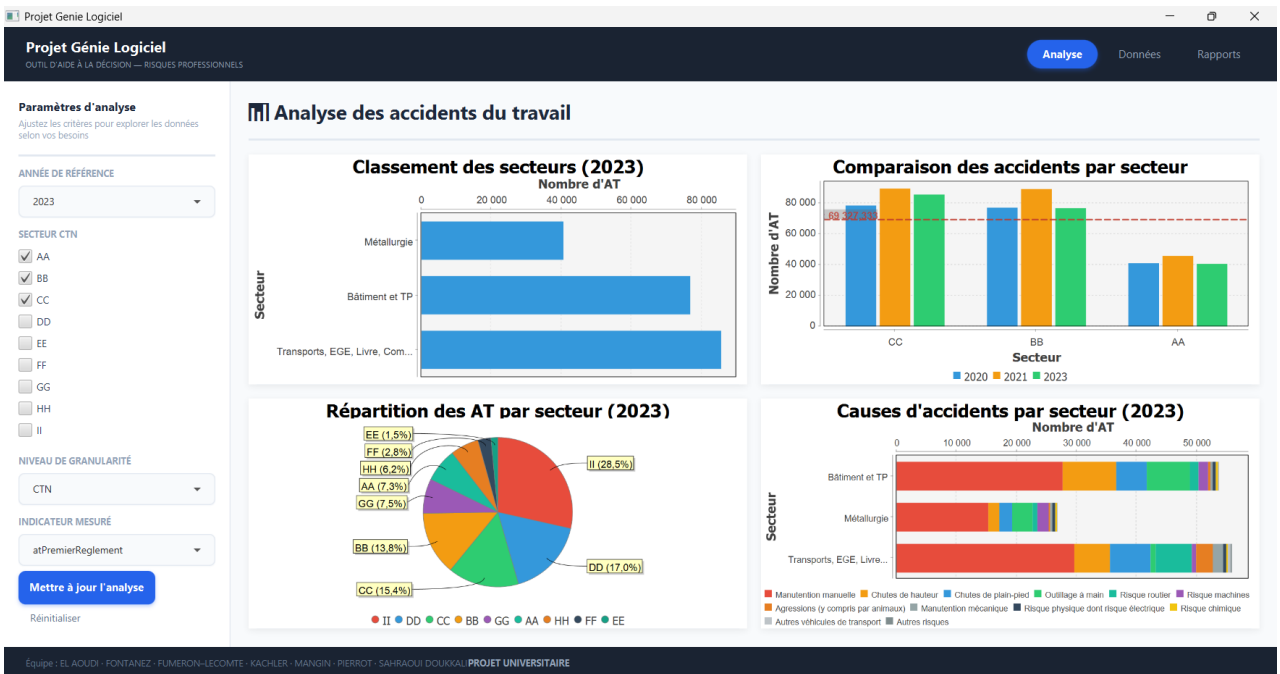


Figure – Maquette Graphiques avancés et comparaison multicritères



Figure 2 - Graphiques avancés et comparaison multicritères



Figure 3 - Maquette Onglet d'information et de contextualisation du projet

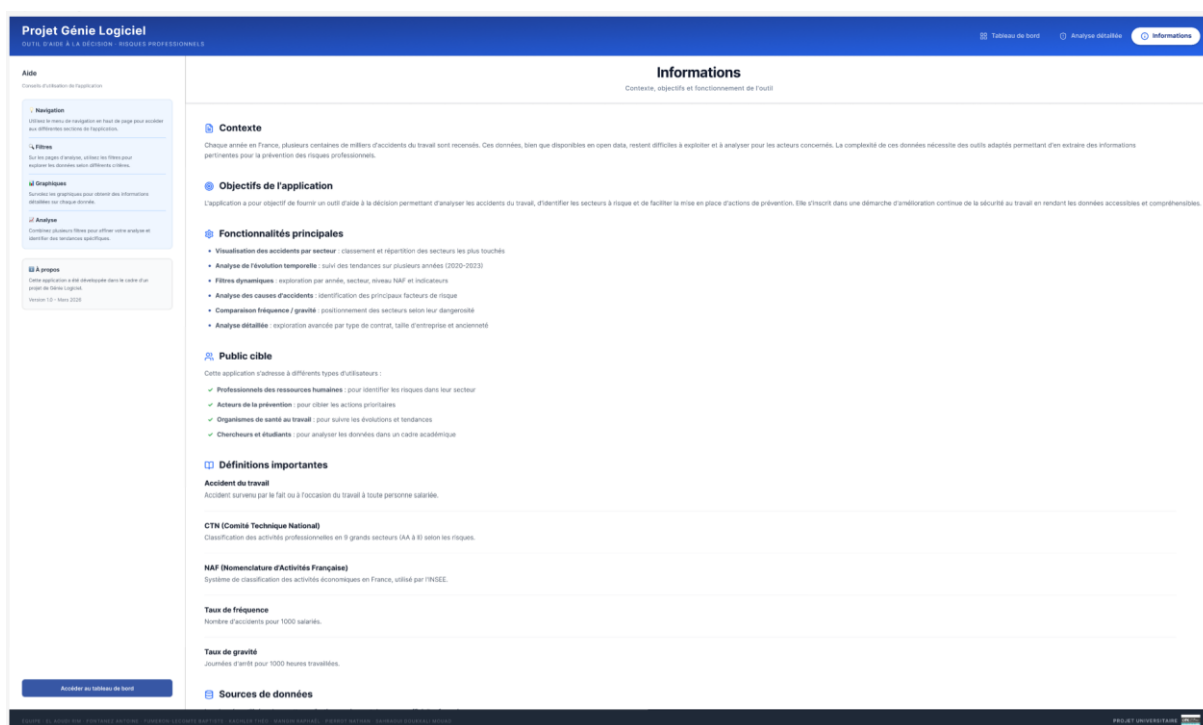
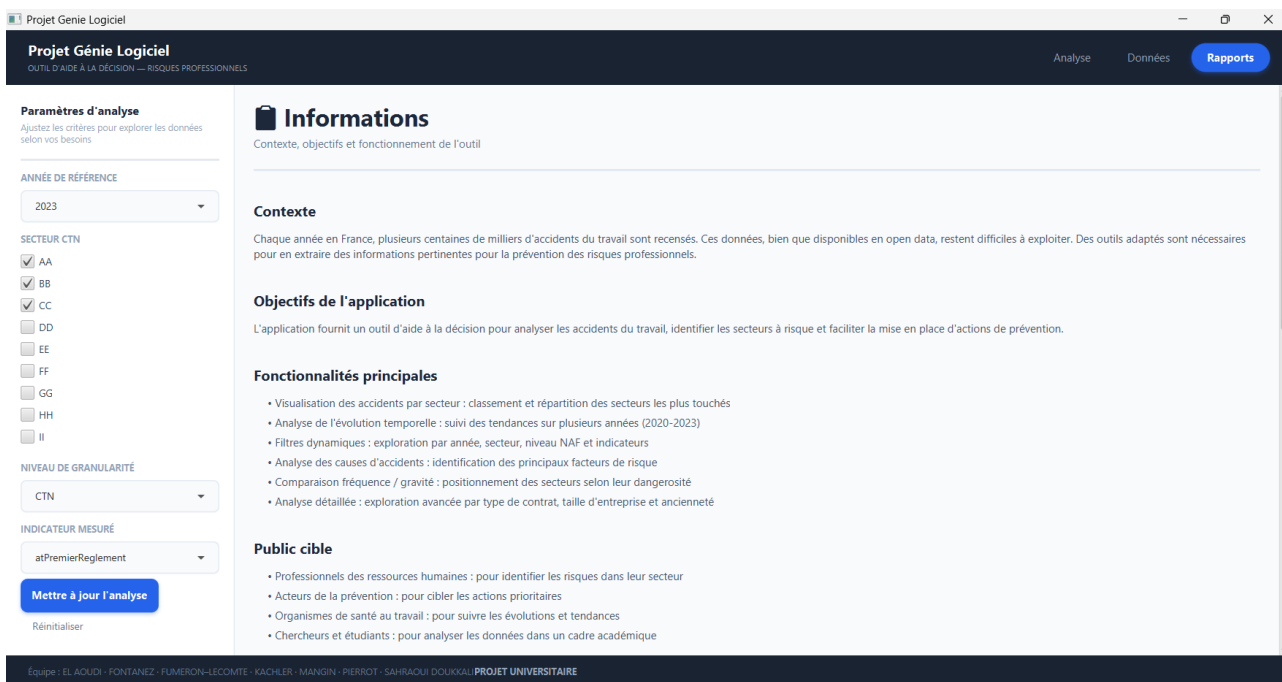


Figure 3 - Maquette Onglet d'information et de contextualisation du projet



Glossaire

Terme	Définition
Accident du travail	Accident survenu par le fait ou à l'occasion du travail
AT	Accident du travail
CTN	Comité Technique National, regroupement de secteurs d'activité
NAF	Nomenclature d'Activités Française
Indice de fréquence	Nombre d'accidents rapporté à 1 000 salariés
Taux de gravité	Nombre de journées d'incapacité temporaire rapporté aux heures travaillées
IT	Incapacité temporaire
IP	Incapacité permanente